

Blockchain and Its Applications

NPTEL Course — IIT Kharagpur

Week 11 In-Depth Study Notes

Lectures 51 – 55

Lecture 51	Hyperledger Aries — Overview, Architecture & Usage
Lecture 52	Blockchain Security I — Common Risks
Lecture 53	Blockchain Security II — Selfish Mining Attack
Lecture 54	Blockchain Security III — Eclipse & Front-Running Attacks
Lecture 55	Use Cases — What Makes a Good Blockchain Use Case?

Keywords

Digital Credentials

Hyperledger Aries

ACA-Py

DIDcomm Protocol

Verifiable Data Registry

Controller

Webhooks

1. What is Hyperledger Aries?

Hyperledger Aries is a project that provides a **shared, reusable, and interoperable toolkit** designed for solutions focused on **creating, transmitting, and storing verifiable digital credentials**. It fills the gap left by Hyperledger Indy, which manages decentralized identities and wallets but does not specify how participants communicate or transfer credentials and presentations to each other.

Key insight: Aries is the *infrastructure* for blockchain-rooted, peer-to-peer interactions. While Indy provides a wallet and verifiable data registry, Aries handles the transport and communication layer between participants (issuer, holder, verifier).

2. Aries Agent Frameworks

Three major implementations of Aries agents exist:

- **Aries Cloud Agent – Python (ACA-Py):** The most popular implementation. Suitable for any non-mobile application and has production deployments.
- **Aries Framework – .NET:** For .NET-based applications.
- **Aries Static Agent – Python:** A lightweight Python agent.

3. ACA-Py Architecture

ACA-Py has three broad modules:

- **Core Capabilities:** Communicates with a distributed ledger (Verifiable Data Registry, e.g., Hyperledger Indy) to resolve DIDs, retrieve credential schemas/definitions, and check revocation lists.
- **Aries Protocols (DIDcomm):** Used for P2P communication with other Aries agents. DIDcomm is transport-independent — it can run over HTTP, TCP, or custom transports.
- **REST API Module:** Exposes an HTTP REST API so a *Controller* can interact with the agent.

The Controller Pattern

Application developers write a **Controller** (not the agent itself) that implements the business logic. The Controller:

- Sends HTTP requests (REST calls) to the ACA-Py agent to execute commands.
- Acts as an **HTTP server** to receive **webhooks** (event notifications) pushed back from the agent — e.g., an incoming connection request from another agent.

Remember: The Aries agent is *stateful*. It maintains connection states, protocol states, and wallet data. Multiple transport types are *pluggable*, as are wallet implementations.

4. Installation (Ubuntu Linux)

```
# Step 1: Add Sovrin repository and install libindy
apt-key adv --keyserver keyserver.ubuntu.com --recv-keys CE7709D068DB5E88
apt-add-repository "deb https://repo.sovrin.org/sdk/deb bionic master" -y
apt-get update && apt-get install -y libindy indy-cli
# Step 2: Install Python SDK for Indy
python3 -m pip install python3-indy
# Step 3: Install ACA-Py
python3 -m pip install aries-cloudagent
# Verify installation
aca-py --version # shows e.g. 0.5.1
aca-py --help
aca-py provision --help
aca-py start --help
```

5. Starting the Aries Agent

Minimum required: at least one inbound and one outbound transport.

```
aca-py start --inbound-transport http 0.0.0.0 8000 --outbound-transport http
```

Full example with Indy + Admin API:

```
aca-py start \
--inbound-transport http 0.0.0.0 8000 \
--outbound-transport http \
--admin-insecure-mode \ # For debugging only
--admin 0.0.0.0 8001 \ # Admin API on port 8001
--seed 00000000000000000000000000000000Steward1 \
--replace-public-did \
--wallet-type indy \
--genesis-file /path/to/pool1.txn \
--wallet-name agent1 \
--wallet-key agent1 \
--log-level debug \
--log-file /tmp/arieslog.txt
```

Key parameters explained:

--seed	A 32-character string used to deterministically generate key pairs. Use the same seed as when the Indy node/steward was created.
--replace-public-did	Ensures the DID derived from the seed becomes the default public DID.
--wallet-type indy	Specifies Hyperledger Indy as the wallet/ledger backend.
--genesis-file	Path to the genesis transaction file of the Indy network pool.
--admin-insecure-mode	Disables API authentication. For experimentation and debugging only — never use in production.
--event-webhook-url	The URL of your controller where the agent will POST event notifications.

6. Admin OpenAPI Interface

When the admin API is enabled, navigate to **http://localhost:8001** to access a Swagger/OpenAPI interface. Key API groups include:

- **server** — /status, /features, /plugins
- **basicmessage** — Send messages over established connections
- **connection** — Create and manage P2P connections between agents
- **issue-credential** — Issue verifiable credentials to holders
- **present-proof** — Request and verify presentations
- **ledger** — Interact with the distributed ledger (e.g., register NYM, get verkey)
- **wallet/did** — Create and list local DIDs; register NYM transactions via steward

Important workflow: A DID created with *wallet/did/create* is only stored in the **local wallet**. To register it on-chain, a steward must issue a **NYM transaction** via *ledger/register-nym*.

Keywords

51% Vulnerability

Private Key Security

Criminal Activities

Double Spending

Transaction Privacy Leakage

1. Taxonomy of Blockchain Risks

Common Risks (Blockchain 1.0 & 2.0)	Specific Risks (Blockchain 2.0 only — Smart Contracts)
• 51% Vulnerability • Private Key Security • Criminal Activity • Double Spending • Transaction Privacy Leakage	• Criminal Smart Contracts • Vulnerabilities in Smart Contracts • Under-optimized Smart Contracts • Under-priced Operations

2. 51% Vulnerability

Blockchain security relies on the difficulty of mining blocks. If an attacker (or pool of attackers) amasses **more than 50% of the total hashing power** (or stake in PoS), they can:

- Mine blocks faster than the rest of the network combined.
- Fork the chain and replace honestly-mined blocks with their own.
- Turn the previously-mined honest blocks into **orphan blocks**, wasting honest miners' resources.
- Effectively make the network **centralized** under their control.

Historical note: There was an incident where a single mining pool approached 50% hash power. The participants voluntarily withdrew resources to maintain decentralization — demonstrating the community self-governance aspect of permissionless blockchains.

3. Private Key Security

All blockchain ownership and signing is based on **private keys**. If an attacker obtains Alice's private key, the attacker can:

- Sign transactions on Alice's behalf — Bob (or any verifier) cannot distinguish between Alice and the attacker.
- Drain all of Alice's cryptocurrency.

Risks of key loss:

- Key theft by an attacker.
- Key loss — forgetting where it was stored.

- Death of the key holder without sharing the key — coins are permanently lost.

Best practice: Store private keys in dedicated hardware security devices (HSMs or hardware wallets). Insert the device only when signing; remove afterwards.

4. Criminal Activities Enabled by Blockchain

Blockchain's anonymity and borderless nature have enabled:

- **Ransomware** (e.g., WannaCry) — attackers encrypt victim files and demand Bitcoin payment; anonymity makes them hard to catch.
- **Underground markets** (e.g., Silk Road) — selling contraband (drugs, weapons) with cryptocurrency payments.
- **Money laundering** — converting unaccounted-for funds into valid coins through the blockchain.

***Risk to blockchain itself:** Governments may restrict/ban cryptocurrency usage if these activities proliferate, undermining the broader ecosystem.*

5. Double Spending

Attack steps:

1. Attacker sends transaction TX_v (to vendor) and TX_a (to colluding address) simultaneously.
2. Through collusion with a mining pool, TX_a is mined into the blockchain first.
3. The vendor, seeing TX_v , provides the service/goods before TX_v is confirmed.
4. TX_v is later detected as a double-spend and rejected.
5. Attacker gets both the goods AND the money (via the colluding address).

Mitigation: Wait for multiple block confirmations before delivering goods/services. The more confirmations, the more expensive it is to undo the transaction.

6. Transaction Privacy Leakage

In Bitcoin, transactions are pseudonymous — addresses are public key hashes, not real names. However, since every transaction input must reference a previous output, a **linkage graph** can be constructed, tracing the chain of custody backwards. If any address in the chain is ever de-anonymized, the entire path is exposed.

Mitigation — Mixins (Chaff Coins):

Used in Cryptonote-based coins. Each transaction input references a set of coins — the real coin plus several fake "chaff" coins (decoys). This makes it hard to determine the true input source.

***Limitation:** Research has shown that many mixins can be eliminated by deduction (the real input is usually the "newest" one), so this is not a complete solution.*

Keywords

Selfish Mining

Attacker's Pool (Private Chain)

Public Chain

Block Suppression

Revenue Manipulation

1. What is Selfish Mining?

Key reference: "Majority Is Not Enough: Bitcoin Mining Is Vulnerable," Ittay Eyal and Emin Güner Sirer, Financial Cryptography 2014.

Selfish mining is an attack where a pool of miners (**less than 50% hash power**) strategically withholds newly mined blocks from the public to build a **private chain**. By selectively publishing private blocks at opportune moments, the attacker:

- Causes honest miners to waste computing resources on blocks that will eventually be orphaned.
- Earns a disproportionately higher revenue share than their hash power would normally allow.
- Makes the attacker's pool appear more profitable, attracting honest miners to join it — increasing the attacker's power over time.

2. Core Mechanism

The attacker maintains a **private branch** (unknown to honest nodes) alongside the **public chain**. The honest nodes keep extending the public chain. The pool keeps extending the private branch. When certain conditions are met, the pool releases some or all of its private blocks, potentially making the private branch the **longest chain** and invalidating honest blocks.

3. The Eight Scenarios

Scenario 1 Any state (except two branches of length 1) — Pool finds a block

Pool appends one block to its private branch, increasing its lead. Revenue from this block is deferred.

Scenario 2 Two branches of length 1 — Pool finds another block

Pool publishes its now-length-2 secret branch. Honest branch (length 1) becomes orphaned. Pool earns revenue of 2.

Scenario 3 Two branches of length 1 — Others find a block after pool's head

Both pool and others earn revenue of 1 each (others for new head, pool for its predecessor).

Scenario 4	Two branches of length 1 — Others find a block after others' head
Others earn revenue of 2; pool earns nothing.	
Scenario 5	No private branch — Others find a block
Others earn revenue of 1. Both pool and others now start mining on the new public head.	
Scenario 6	Lead of 1 — Others find a block
Pool immediately publishes its single secret block. Now two branches of length 1 exist. Revenue is undetermined — governed by later scenarios.	
Scenario 7	Lead of 2 — Others find a block
Pool publishes all its secret blocks (length 3). Honest branch (length 2) becomes orphaned. Pool earns revenue of 2; everyone mines on pool's new head.	
Scenario 8	Lead > 2 — Others find a block
Pool reveals only its i-th block, maintaining a cushion of ≥ 2 . Pool earns revenue of 1. This frustrates honest miners and entices them to join the pool.	

Why it works: The attacker doesn't need >50% power. By strategically timing block releases, the attacker earns more revenue per unit of hash power than honest miners. Over time, honest miners migrate to the attacker's pool, compounding the advantage.

Keywords

Eclipse Attack

Peer-to-Peer Network

IP Table Manipulation

Front-Running Attack

Displacement

Insertion

Suppression

1. Eclipse Attack

Key reference: "Eclipse Attacks on Bitcoin's Peer-to-Peer Network," Heilman et al., USENIX Security 2015.

Unlike most other attacks (which target the application/consensus layer), the Eclipse Attack targets the **peer-to-peer network layer**. The goal is to **isolate a victim node** from the rest of the network by ensuring all of the victim's connections lead only to attacker-controlled nodes.

How the attack works:

Step 1 — Populate Peer Tables: Bitcoin nodes maintain two IP tables:

- *New Table:* IPs the node has heard about but not yet connected to.
- *Tried Table:* IPs the node has previously connected to.

Each IP entry has a timestamp. New connections prefer IPs with the newest timestamps.

Step 2 — Selection Bias: Attacker floods the tried table with its own IPs (with fresh timestamps) and fills the new table with junk IPs. After restart, the victim always picks attacker IPs.

Step 3 — Trigger a Restart: Restarts happen naturally (software updates, DoS, power failure, ISP outages). After restart, victim connects exclusively to attacker nodes.

Step 4 — Bucket Eviction (Eviction Bias): When an IP bucket is full and a new IP must be inserted, the node randomly selects 4 IPs and deletes the oldest. Attacker keeps refreshing its IPs' timestamps, ensuring honest IPs are evicted over time.

What can an attacker do once a node is eclipsed?

- Control which part of the blockchain state the victim sees.
- Selectively drop or delay transactions from the victim.
- Execute a **51% attack with less than 51% hash power** — by partitioning miners so each partition has less hash power than the attacker.

Example: If the victim has 30% hash power and the rest of the network has 30%, the attacker (40%) outcompetes each isolated party ($30\% < 40\%$), even though together the honest miners (60%) would beat the attacker.

2. Front-Running Attack

Key reference: "SoK: Transparent Dishonesty: Front-Running Attacks on Blockchain," Eskandari et al., FC 2019.

Front-running occurs when an attacker observes a pending transaction in the mempool and exploits foreknowledge to get their own transaction executed first — to financial advantage. This exploits the fact that **miners choose transaction ordering** and users can see unconfirmed transactions.

Basic example:

Alice sends Buy(1000 ETH) with gasPrice = 5 Gwei. The attacker sees this and sends their own Buy(1000 ETH) with gasPrice = 50 Gwei. Miners include the attacker's transaction first (higher gas), the price rises, and now Alice must buy at the inflated price.

Three types of front-running:

Attacker's transaction makes Alice's transaction **irrelevant**. Alice's action is not needed afterwards. Example: Attacker buys a domain name that Alice is about to purchase — Alice's transaction fails.

Attacker's transaction changes the state, forcing Alice's transaction to execute in a **different (worse) state**. Example: Attacker front-runs Alice's large ETH purchase, driving up the price so Alice pays more.

Attacker delays Alice's transaction until it no longer matters. Example: The Fomo3D game — attacker floods the network with its own DApp calls to block Alice's ticket purchase until the game timer expires.

Two additional sub-categories:

- **Asymmetric:** Alice tries to cancel an offer; attacker front-runs the cancellation to fulfill the offer first.
- **Bulk:** Attacker runs a large set of transactions that move prices, forcing Alice to transact at a disadvantageous state.

The Fomo3D Suppression Example (Real-World Case)

Fomo3D was an Ethereum game: the last person to buy a ticket before a timer expired won the jackpot. Each ticket purchase extended the timer by ~30 seconds. Theoretically, the game would run forever. A winner exploited suppression:

1. Bought 1 ticket when ~3 minutes remained.
2. Immediately flooded the network by sending many high-gasPrice transactions to their own DApps.
3. Network congestion meant no other ticket-purchase transaction could be included in a block before the timer expired.
4. The winner's ticket purchase was the last — they won the jackpot.

Why this is possible: Blockchain is decentralized — there is no central authority to prevent or penalize front-running. Unlike traditional financial markets where front-running is illegal, in a permissionless blockchain, there is no recourse.

Keywords

Use Cases for Enterprises
Business Problem
Business Network
Participants
Assets
Transactions
Trust
Incentive

1. Industry Use Cases (Overview)

Financial Services	Trade finance (TradeLens), cross-currency payments, mortgages, KYC, cross-border tax
Public Sector	Asset/land registration, citizen identity management (SSI/DIDs), medical records, medicine supply chain
Retail	Supply chain (IBM Food Trust), loyalty programs, supplier-retailer information sharing
Insurance	Claims processing, risk provenance, asset usage history, claims file
Supply Chain & Logistics	Supply chain finance, maintenance tracking, provenance, supply chain compliance

2. What Makes a Good Blockchain Use Case?

Three fundamental requirements must all be present:

Requirement 1: A Business Problem

The problem cannot be solved more efficiently with a simpler or existing technology. The overhead of blockchain must be justified by benefits.

Requirement 2: An Identifiable Business Network

Must involve multiple participants, assets, and transactions. A single organization under one authority does NOT need blockchain — use a centralized database instead.

Requirement 3: A Need for Trust

The participants cannot (or should not need to) blindly trust each other or a central third party. Blockchain provides trust through: Consensus, Immutability, Finality, and Provenance.

3. Checklist: Understanding the Business Problem

- What is the specific business problem/challenge?
- What is the current way of solving it (existing systems/processes)?
- Which specific aspects of the larger problem will this solution address?

4. Checklist: Understanding the Participants

- Who are the organizations and what are their roles?
- Who are the specific users (people) and their job roles?
- How will participants access and interact with the blockchain (web app, mobile, peer node)?
- Are gateways or external data integrations needed?
- Who will operate, govern, and regulate the blockchain?
- What is the value/incentive for each participant to join?
- Do you need to know your users' identities? (PoI vs pseudo-anonymity)

5. Checklist: Understanding Assets & Transactions

- What assets are involved and what key information is associated with them?
- Are the assets publicly verifiable? (If not, blockchain adds no value)
- What transactions are needed: add, delete, change, transfer, query?
- Under what business/contractual conditions do assets transfer between owners?
- Should access to certain transactions be role-based?

6. Additional Considerations

- What are the main workflow steps and who executes them?
- What is the expected benefit for each participant? Is the benefit equitable?
- What legacy systems need to be integrated and how deeply?

Common Anti-Patterns to Avoid

- **Sensor data on blockchain** without validation logic — unverifiable data has no value in a blockchain.
- **All IoT devices in one home network** on a blockchain — single-party control means no need for decentralization.
- **Government hospitals sharing data** — if all parties trust the government as a central authority, a centralized database is simpler and sufficient.
- **Blockchain for everything** — centralized solutions with replication/backups address "single point of failure" arguments.

The trade-off principle: Always ask — does the overhead of blockchain (complexity, performance, cost) outweigh the benefit it provides? If yes, choose a simpler alternative.

Week 11 — Quick Reference Summary

Topic	Core Concept
-------	--------------

Hyperledger Aries	P2P infrastructure for digital credential transfer; uses DIDcomm protocol; controlled via REST API controller pattern
ACA-Py	Most popular Aries agent; pluggable transports & wallets; driven by controller via REST + webhooks
51% Vulnerability	Attacker with >50% hash power controls block generation and can orphan honest blocks
Private Key Security	Entire ownership rests on private key — theft or loss means permanent loss of assets
Double Spending	Colluding with a mining pool to confirm a fraudulent transaction before the vendor detects it
Transaction Privacy	Linkage graph from UTXO chain reveals spending history; mixins partially obscure but not eliminate this
Selfish Mining	Withhold mined blocks to build a private lead; release strategically to orphan honest blocks and gain disproportionate revenue
Eclipse Attack	Populate victim's peer tables with attacker IPs; isolate victim to enable 51% attack with less hash power
Front-Running	Exploit mempool visibility to execute transactions ahead of victims; three types: displacement, insertion, suppression
Good Use Case	Must have: (1) business problem, (2) multi-party business network, (3) genuine need for trust/decentralization